

► EXTERNAL TOOL ARSENAL & VAPT CYCLE MAPPING

ARSENAL

► 72 tools catalogued | 23 categories | 5 VAPT phases

External Tool Arsenal & VAPT Cycle Reference

A personal, hands-on reference for the 72 external command-line tools I use across reconnaissance, scanning, vulnerability assessment, exploitation, and mobile security testing. Each tool is mapped to the stage of the Vulnerability Assessment & Penetration Testing (VAPT) lifecycle it serves, with a plain-language rationale for why it earns a place in my toolkit and a representative command showing how it's actually invoked.

DOCUMENT SUBJECT	Personal VAPT Tool Arsenal & Reference				
TOOLS CATALOGUED	72 (across 23 categories)				
VAPT PHASES MAPPED	5 — Recon, Scanning, Assessment, Exploitation, Mobile/Post-Exploitation				
REFERENCE DATE	July 2026				
AUTHOR	Muhammad Huzaifa Jamil				
TITLE	Cyber Security (SW) Engineer				
CONTACT	m.huzaifa.jamil@outlook.com · m.huzaifa.jamil.cys@gmail.com				
DOCUMENT VERSION	1.0 — Portfolio Edition				
26 PHASE 1 25 PHASE 2 8 PHASE 3 11 PHASE 4 2 PHASE 5 72 TOTAL TOOLS					

► FOR NON-TECHNICAL READERS — WHAT THIS DOCUMENT IS

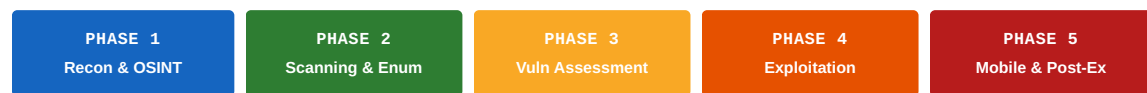
WHAT IS A VAPT CYCLE?

A Vulnerability Assessment & Penetration Test (VAPT) is not one activity — it is a sequence of stages, each building on the last: find what exists, map how it responds, check it against known weaknesses, prove exploitability, and (for mobile apps) get inside the running application itself. This document groups every tool I use by which of those five stages it belongs to.

WHY SO MANY SEPARATE TOOLS?

No single tool does all five stages well — each one specializes (subdomain discovery, port scanning, injection testing, credential attacks, mobile instrumentation) the same way a physical security audit uses different specialists for locks, alarms, and safes. **72 tools across 23 categories** give me professional-grade coverage at every stage instead of relying on one generalist tool to do all of it poorly.

WHAT DO THE PHASE COLOURS MEAN?



Work generally flows left to right — Recon feeds Scanning, Scanning feeds Assessment, Assessment feeds Exploitation — though in practice a real engagement loops back and forth between stages as new information surfaces.

WHAT'S THE BOTTOM LINE?

Every tool in this catalogue is one I have personally used hands-on across reconnaissance, scanning, assessment, and exploitation work — this is a working reference built from practice, not a theoretical wishlist.

01 EXECUTIVE SUMMARY

This document provides a complete inventory of the **72 external command-line tools** in my personal security-testing toolkit, spanning **23 functional categories** from passive subdomain enumeration through mobile runtime instrumentation. Every tool is mapped to exactly one of the **5 stages of the VAPT lifecycle** it primarily serves, alongside a plain-language rationale for why it earned a place in the toolkit and a representative command demonstrating real-world usage.

This is a living reference, not a static list — each entry reflects a tool I have actually run against real targets, with the command syntax and rationale drawn from that hands-on use rather than copied from a README.

HOW TO READ THIS DOCUMENT

§02 gives the full flat inventory as a single scannable table. §03 breaks down tool counts and the phase model itself. §04–08 give one detailed card per tool, grouped by the VAPT phase it belongs to, each with a "why" rationale and an example invocation.

The table below lists all 72 tools in phase order (Recon → Scanning → Assessment → Exploitation → Mobile/Post-Exploitation). Full rationale and example commands for each tool are in the phase-detail sections that follow (§04–08).

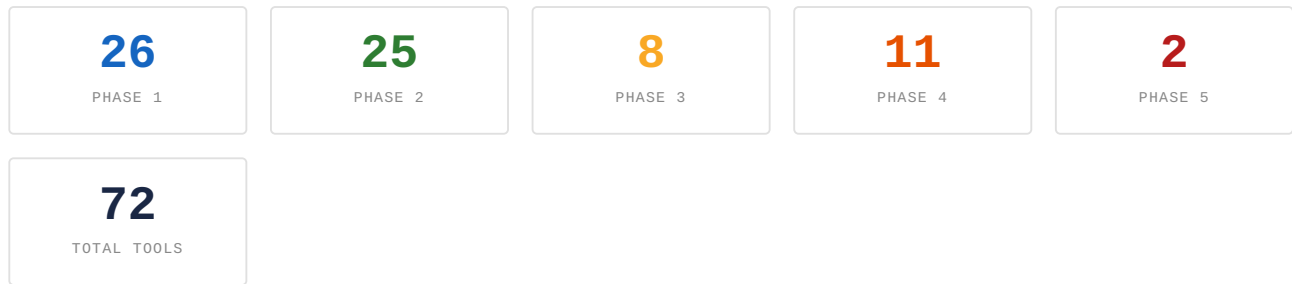
TOOL	CATEGORY	VAPT PHASE	STATUS	WHY IT'S USED
subfinder	RECON	PHASE 1	INSTALLED	Fastest passive subdomain enumeration tool, pulling from 20+ public sources (CT logs, DNS aggregators, search APIs).
amass	RECON	PHASE 1	INSTALLED	Deep passive-and-active recon: DNS brute-forcing, ASN/network mapping, and a relationship graph.
assetfinder	RECON	PHASE 1	INSTALLED	Lightweight secondary passive source chained into the subdomain pipeline to catch names the primary tools miss.
bbot	RECON	PHASE 1	INSTALLED	Modular OSINT/recon automation framework chaining 80+ modules.
theHarvester	RECON	PHASE 1	INSTALLED	OSINT harvester for employee emails, names, and hosts from search engines and CT logs.
dnsrecon	RECON	PHASE 1	INSTALLED	DNS enumeration and hygiene check.
massdns	RECON	PHASE 1	INSTALLED	High-performance DNS resolver capable of resolving millions of hostnames per minute.
puredns	RECON	PHASE 1	INSTALLED	Wildcard-aware DNS brute-force/resolution wrapper around massdns.
shuffledns	RECON	PHASE 1	INSTALLED	ProjectDiscovery's massdns wrapper for large-scale subdomain resolving and brute forcing.
knockpy	RECON	PHASE 1	INSTALLED	Standalone subdomain brute forcer with a built-in wordlist and basic takeover fingerprint check.
sublert	RECON	PHASE 1	INSTALLED	Continuous subdomain monitor driven by Certificate Transparency log streams.
maigret	RECON	PHASE 1	INSTALLED	Username OSINT across 3000+ sites, used to pivot from a leaked handle to other accounts or exposed personal info once an employee identity surfaces during recon.
pywhat	RECON	PHASE 1	INSTALLED	Instant string identifier.
s3scanner	CLOUD	PHASE 1	INSTALLED	Enumerates AWS S3 (and compatible) buckets tied to a target/keyword and checks their permissions.
cloud_enum	CLOUD	PHASE 1	INSTALLED	Multi-cloud public resource enumerator.
cloudfail	CLOUD	PHASE 1	INSTALLED	Locates the real origin server behind a Cloudflare-fronted target via historical DNS and subdomain misconfig.
scoutsuite	CLOUD	PHASE 1	INSTALLED	Multi-cloud security posture auditor (AWS/Azure/GCP/Alibaba) for misconfigured IAM, storage, and networking.
trufflehog	SECRETS	PHASE 1	INSTALLED	Deep secret scanner that actively verifies each candidate credential against its issuer's API.
noseyparker	SECRETS	PHASE 1	INSTALLED	Extremely fast secret scanner purpose-built for huge git histories.
gitleaks	SECRETS	PHASE 1	INSTALLED	Git-native secret scanner with a large, low-false-positive default rule pack.
shhgit	SECRETS	PHASE 1	INSTALLED	Real-time GitHub/GitLab/Bitbucket public-commit watcher.

TOOL	CATEGORY	VAPT PHASE	STATUS	WHY IT'S USED
git-hound	SECRETS	PHASE 1	INSTALLED	GitHub code-search-based secret hunter with regex and entropy scoring.
apkleaks	MOBILE	PHASE 1	INSTALLED	First tool run against any Android APK.
jadx	MOBILE	PHASE 1	INSTALLED	Java/Android decompiler producing readable source from an APK/DEX.
linkfinder	JS	PHASE 1	INSTALLED	Extracts endpoint paths and URLs from JavaScript source and minified bundles.
bbscope	SCOPE	PHASE 1	INSTALLED	Pulls every in-scope asset for a program across HackerOne/Bugcrowd/Intigriti/YesWeHack/Immunefi.
httpx	PROBE	PHASE 2	INSTALLED	Fast HTTP toolkit: live-host probing, tech-stack fingerprinting, status codes, and page titles.
dnsx	PROBE	PHASE 2	INSTALLED	Fast DNS resolution/enumeration toolkit that filters wildcard-DNS false positives out of huge subdomain lists before they reach httpx.
naabu	PROBE	PHASE 2	INSTALLED	Fast port scanner built for recon pipelines.
smap	PROBE	PHASE 2	INSTALLED	Nmap-compatible port scanner that queries Shodan's index instead of scanning directly.
aquatone	PROBE	PHASE 2	INSTALLED	Screenshots every live host for rapid visual triage of a large asset list.
eyewitness	PROBE	PHASE 2	INSTALLED	Alternative screenshot and reporting tool that also captures response headers/certs.
katana	CRAWL	PHASE 2	INSTALLED	Modern web crawler with JS-rendering support.
gau	CRAWL	PHASE 2	INSTALLED	Pulls known URLs for a domain from the Wayback Machine, Common Crawl, AlienVault OTX, and URLScan.
waybackurls	CRAWL	PHASE 2	INSTALLED	Focused single-source Wayback Machine CDX puller.
waymore	CRAWL	PHASE 2	INSTALLED	Extended Wayback/CDX harvester that also retrieves archived response bodies.
hakrawler	CRAWL	PHASE 2	INSTALLED	Fast, simple crawler for a quick pass on rate-limited targets, where katana's heavier JS rendering would trip WAF rate thresholds.
gospider	CRAWL	PHASE 2	INSTALLED	Crawler that also mines robots.txt/sitemap.xml and cross-domain links, filling gaps katana/hakrawler occasionally miss.
cariddi	CRAWL	PHASE 2	INSTALLED	Crawler that flags secrets, endpoints, and juicy file extensions inline while crawling.
ffuf	FUZZ	PHASE 2	INSTALLED	Fast, flexible HTTP fuzzer.
feroxbuster	FUZZ	PHASE 2	INSTALLED	Rust-based recursive content-discovery fuzzer.
gobuster	FUZZ	PHASE 2	INSTALLED	Long-standing standard directory/DNS/vhost bruteforcer.
arjun	PARAM	PHASE 2	INSTALLED	Discovers hidden/unlinked HTTP GET/POST parameters via wordlist probing.
x8	PARAM	PHASE 2	INSTALLED	Rust alternative to Arjun for hidden-parameter discovery, used when Python performance is too slow against a large parameter wordlist or many target URLs.
gf	FILTER	PHASE 2	INSTALLED	Pattern-matching grep wrapper with curated vulnerability-class patterns (XSS, SSRF, SQLi sinks).

TOOL	CATEGORY	VAPT PHASE	STATUS	WHY IT'S USED
qsreplace	FILTER	PHASE 2	INSTALLED	Replaces query-string values across a list of URLs with a payload placeholder.
anew	FILTER	PHASE 2	INSTALLED	Appends only new/unique lines to a file.
dnsreaper	TAKEOVER	PHASE 2	INSTALLED	Subdomain-takeover scanner supporting 50+ provider fingerprints.
subjack	TAKEOVER	PHASE 2	INSTALLED	Fast Go-based takeover fingerprint scanner used as a lightweight fallback when dnsreaper's fingerprint database doesn't cover a specific provider.
graphw00f	GRAPHQL	PHASE 2	INSTALLED	Fingerprints which GraphQL engine (Apollo, Hasura, Graphene, etc.) a target endpoint runs.
clairvoyance	GRAPHQL	PHASE 2	INSTALLED	Reconstructs a GraphQL schema via field-suggestion error messages even when introspection is disabled.
nuclei	SCAN	PHASE 3	INSTALLED	Template-driven vulnerability scanner covering thousands of CVEs, misconfigs, and exposed panels.
semgrep	SAST	PHASE 3	INSTALLED	Static analysis engine that flags insecure code patterns across languages via rulesets.
log4j-scan	CVE	PHASE 3	INSTALLED	Focused active scanner for Log4Shell (CVE-2021-44228) and its bypass variants.
graphql-cop	GRAPHQL	PHASE 3	INSTALLED	Automated GraphQL security checklist covering the OWASP GraphQL testing guide (introspection, batching, field suggestion, CSRF).
jwt_tool	JWT	PHASE 3	INSTALLED	Comprehensive JWT security tester.
byp4xx	BYPASS	PHASE 3	INSTALLED	Automated 403/401 bypass tester across header, path-encoding, and method tricks.
whatwaf	BYPASS	PHASE 3	INSTALLED	Detects and fingerprints which WAF/CDN protects a target, so encoding and evasion payloads can be chosen for that specific vendor instead of guessed blind.
unwaf	BYPASS	PHASE 3	INSTALLED	Resolves the real origin IP behind a WAF/CDN using historical DNS and certificate data.
dalfox	XSS	PHASE 4	INSTALLED	Fast XSS scanner with parameter mining, DOM analysis, and blind-XSS support.
xsstrike	XSS	PHASE 4	INSTALLED	Context-aware XSS detection/exploitation engine with WAF fingerprinting.
ghauri	SQLI	PHASE 4	INSTALLED	Modern SQL injection tool built for speed on boolean/time-based blind SQLi.
sqlmap	SQLI	PHASE 4	INSTALLED	The standard automated SQL injection framework.
fuxploader	UPLOAD	PHASE 4	INSTALLED	Automated file-upload vulnerability scanner/exploiter.
hashcat	CRED	PHASE 4	INSTALLED	GPU-accelerated password/hash cracking engine.
cewler	CRED	PHASE 4	INSTALLED	Crawls a target site to build a custom wordlist from its own on-page vocabulary.
cupp	CRED	PHASE 4	INSTALLED	Interactive profiler that turns known facts about a specific employee (name, birthdate, pet, company) into a small high-probability password candidate list for a targeted spray.

TOOL	CATEGORY	VAPT PHASE	STATUS	WHY IT'S USED
trevorspray	CRED	PHASE 4	INSTALLED	Password-spray orchestrator with OAuth/O365/Okta module support and jitter/delay controls.
kerbrute	CRED	PHASE 4	INSTALLED	Fast Kerberos pre-auth username enumeration and password spraying.
interactsh-client	OOB	PHASE 4	INSTALLED	Self-hosted out-of-band interaction server client.
mobsf	MOBILE	PHASE 5	INSTALLED	All-in-one mobile app security framework running automated static and dynamic analysis in one shot.
objection	MOBILE	PHASE 5	INSTALLED	Runtime mobile instrumentation toolkit built on Frida.

My toolkit is organised around a standard 5-stage penetration-testing lifecycle. Tool counts per phase are shown below; the detailed rationale for each phase's tools follows in §04–08.



- **Phase 1 — Reconnaissance & OSINT (26 tools):** Passive and low-noise active discovery of everything that belongs to the target before a single test payload is sent — subdomains, cloud storage, leaked secrets, employee identities, and mobile app internals.
- **Phase 2 — Scanning & Enumeration (25 tools):** Converting the recon asset list into a concrete map of live services, open ports, endpoints, parameters, and API surface — the input every later phase tests against.
- **Phase 3 — Vulnerability Assessment (8 tools):** Automated and semi-automated checks against the mapped surface for known CVEs, misconfigurations, weak authentication tokens, and WAF posture — triage before manual exploitation effort is spent.
- **Phase 4 — Exploitation (11 tools):** Turning a suspected weakness into a proven, reproducible impact — injection, XSS, file-upload RCE, credential attacks, and blind-vulnerability confirmation via out-of-band callbacks.
- **Phase 5 — Mobile Runtime & Post-Exploitation (2 tools):** Runtime instrumentation of mobile applications once static analysis is exhausted — defeating SSL pinning and driving live traffic capture for the same API-testing playbook used against the web target.

NOTE ON DUAL-USE TOOLS

Several tools genuinely span two phases — `gf/qsreplace/anew` are enumeration-support utilities also used to feed exploitation pipelines, and `jadx/apkleaks` perform static mobile recon that later informs the Phase 5 runtime work. Each tool below is assigned to the phase where it is *first* and most characteristically used, to keep the inventory a strict partition rather than an overlapping one.

PHASE 1 — RECONNAISSANCE & OSINT (26 TOOLS)

Passive and low-noise active discovery of everything that belongs to the target before a single test payload is sent — subdomains, cloud storage, leaked secrets, employee identities, and mobile app internals.

subfinder

github.com/projectdiscovery/subfinder

RECON

INSTALLED

Fastest passive subdomain enumeration tool, pulling from 20+ public sources (CT logs, DNS aggregators, search APIs). The first command run against any new target — maps the *.target.com surface without a single packet touching target infrastructure.

```
$ subfinder -d target.com -all -silent -o subs.txt
```

amass

github.com/owasp-amass/amass

RECON

INSTALLED

Deep passive-and-active recon: DNS brute-forcing, ASN/network mapping, and a relationship graph. Complements subfinder when passive sources miss internal or legacy subdomains.

```
$ amass enum -d target.com -passive -o amass_subs.txt
```

assetfinder

github.com/tomnomnom/assetfinder

RECON

INSTALLED

Lightweight secondary passive source chained into the subdomain pipeline to catch names the primary tools miss — cheap insurance, near-zero runtime cost.

```
$ assetfinder --subs-only target.com
```

bbot

github.com/blacklanternsecurity/bbot

RECON

INSTALLED

Modular OSINT/recon automation framework chaining 80+ modules. One-shot recon automation for large-scope programs — subdomain enum, cloud enum, and secret scanning in a single graph-based run.

```
$ bbot -t target.com -f subdomain-enum cloud-enum -o bbot_out/
```

theHarvester

github.com/laramies/theHarvester

RECON

INSTALLED

OSINT harvester for employee emails, names, and hosts from search engines and CT logs — the first stage of any credential-attack pipeline, building the target list before a spray.

```
$ theHarvester -d target.com -b all -f harvest.json
```

dnsrecon

github.com/darkoperator/dnsrecon

RECON

INSTALLED

DNS enumeration and hygiene check — zone-transfer attempts, full record dumps, and wordlist brute force, used as a cross-check against SPF/DMARC and delegation misconfigs.

```
$ dnsrecon -d target.com -t std,brt
```

massdns

github.com/blechschmidt/massdns

RECON

INSTALLED

High-performance DNS resolver capable of resolving millions of hostnames per minute — the resolution engine underneath puredns/shuffledns brute-force enumeration.

```
$ massdns -r resolvers.txt -t A -o S wordlist_subs.txt
```

puredns

github.com/d3mondev/puredns

RECON

INSTALLED

Wildcard-aware DNS brute-force/resolution wrapper around massdns — filters out wildcard false positives that plain massdns brute forcing would otherwise report as live hosts.

```
$ puredns bruteforce wordlist.txt target.com -r resolvers.txt
```

shuffledns

github.com/projectdiscovery/shuffledns

RECON

INSTALLED

ProjectDiscovery's massdns wrapper for large-scale subdomain resolving and brute forcing — used interchangeably with puredns depending on wordlist size and resolver pool health.

```
$ shuffledns -d target.com -w wordlist.txt -r resolvers.txt -mode bruteforce
```

knockpy

github.com/guelfoweb/knockpy

RECON

INSTALLED

Standalone subdomain brute forcer with a built-in wordlist and basic takeover fingerprint check — a quick single-command alternative when a heavier pipeline isn't warranted.

```
$ knockpy target.com
```

sublert

github.com/yassineaboukir/sublert

RECON

INSTALLED

Continuous subdomain monitor driven by Certificate Transparency log streams — catches newly-issued certificates for target.com the moment they go live, ahead of other hunters.

```
$ sublert -u target.com -m
```

maigret

github.com/soxoj/maigret

RECON

INSTALLED

Username OSINT across 3000+ sites, used to pivot from a leaked handle to other accounts or exposed personal info once an employee identity surfaces during recon.

```
$ maigret johndoe --json simple
```

pywhat

github.com/bee-san/pywhat

RECON

INSTALLED

Instant string identifier — tells you whether a string pulled from a leak is a JWT, AWS key, private key, or crypto address before deeper manual investigation begins.

```
$ pywhat 'AKIAIOSFODNN7EXAMPLE'
```

s3scanner

github.com/sa7mon/S3Scanner

CLOUD

INSTALLED

Enumerates AWS S3 (and compatible) buckets tied to a target/keyword and checks their permissions — publicly-readable or writable buckets are among the highest-yield, lowest-effort vulnerability classes in independent security research.

```
$ s3scanner scan -bucket-file candidate_buckets.txt
```

cloud_enum

github.com/initstring/cloud_enum

CLOUD

INSTALLED

Multi-cloud public resource enumerator — extends bucket hunting beyond AWS to Azure Blob Storage and GCP buckets for the same keyword in a single pass.

```
$ cloud_enum -k target -l cloud_enum_results.txt
```

cloudfail

github.com/m0rtem/CloudFail

CLOUD

INSTALLED

Locates the real origin server behind a Cloudflare-fronted target via historical DNS and subdomain misconfig — useful for testing without WAF interference, sometimes a finding on its own if the origin accepts direct connections.

```
$ cloudfail -t target.com
```

scoutsuite

github.com/nccgroup/ScoutSuite

CLOUD

INSTALLED

Multi-cloud security posture auditor (AWS/Azure/GCP/Alibaba) for misconfigured IAM, storage, and networking — used on engagements that grant read-only cloud-account access.

```
$ scout aws --profile target-readonly
```

trufflehog

github.com/trufflesecurity/trufflehog

SECRETS

INSTALLED

Deep secret scanner that actively verifies each candidate credential against its issuer's API — separates theoretical leaks in git history/JS bundles from ones that are actually live.

```
$ trufflehog git https://github.com/target/repo --only-verified
```

noseyparker

github.com/praetorian-inc/noseyparker

SECRETS

INSTALLED

Extremely fast secret scanner purpose-built for huge git histories — used in place of trufflehog when raw scan speed matters more than live credential verification.

```
$ noseyparker scan --datastore np.db /path/to/repo
```

gitleaks

github.com/gitleaks/gitleaks

SECRETS

INSTALLED

Git-native secret scanner with a large, low-false-positive default rule pack — the default first-pass scan on any in-scope repo or CI history, including CI/CD security assessments.

```
$ gitleaks detect --source . --report-path gitleaks.json
```

shhgit

github.com/eth0izzle/shhgit

SECRETS

INSTALLED

Real-time GitHub/GitLab/Bitbucket public-commit watcher — catches secrets the instant a developer accidentally pushes them for the target's org or keyword.

```
$ shhgit --search-query target-org
```

git-hound

github.com/tillson/git-hound

SECRETS

INSTALLED

GitHub code-search-based secret hunter with regex and entropy scoring — reaches secrets via keyword search across all of GitHub, not just repos you already know to clone.

```
$ githound --query target.com --dorks dorks.txt
```

apkleaks

github.com/dwiswant0/apkleaks

MOBILE

INSTALLED

First tool run against any Android APK — scans the compiled package for hardcoded API keys, URLs, and secrets before deeper decompilation is needed.

```
$ apkleaks -f app.apk -o apkleaks_out.txt
```

jadx

github.com/skylot/jadx

MOBILE

INSTALLED

Java/Android decompiler producing readable source from an APK/DEX — used when apkleaks' pattern scan isn't enough and you need to read the actual app logic, hidden endpoints, or SSL-pinning implementation.

```
$ jadx -d jadx_out app.apk
```

linkfinder

github.com/GerbenJavado/LinkFinder

JS

INSTALLED

Extracts endpoint paths and URLs from JavaScript source and minified bundles — run against every JS file collected while crawling; regularly surfaces internal API paths never linked from the visible UI.

```
$ linkfinder -i bundle.js -o results.html
```

bbscope

github.com/sw33tLie/bbscope

SCOPE

INSTALLED

Pulls every in-scope asset for a program across HackerOne/Bugcrowd/Intigriti/YesWeHack/Immunefi. The literal first command of any engagement — the authoritative host list every later command is filtered against.

```
$ bbscope h1 -o "target-handle" -b
```

PHASE 2 — SCANNING & ENUMERATION (25 TOOLS)

Converting the recon asset list into a concrete map of live services, open ports, endpoints, parameters, and API surface — the input every later phase tests against.

httpx

github.com/projectdiscovery/httpx

[PROBE](#)[INSTALLED](#)

Fast HTTP toolkit: live-host probing, tech-stack fingerprinting, status codes, and page titles. Converts a raw subdomain list into confirmed-alive web services — the pivot point between recon and every later phase.

```
$ cat subs.txt | httpx -silent -title -tech-detect -status-code -o live.txt
```

dnsx

github.com/projectdiscovery/dnsx

[PROBE](#)[INSTALLED](#)

Fast DNS resolution/enumeration toolkit that filters wildcard-DNS false positives out of huge subdomain lists before they reach httpx.

```
$ dnsx -l subs.txt -a -resp -o resolved.txt
```

naabu

github.com/projectdiscovery/naabu

[PROBE](#)[INSTALLED](#)

Fast port scanner built for recon pipelines — sweeps every live host for non-standard admin panels, dev servers, and exposed databases a web-only scan would miss.

```
$ naabu -l hosts.txt -top-ports 1000 -o open_ports.txt
```

smap

github.com/s0md3v/Smmap

[PROBE](#)[INSTALLED](#)

Nmap-compatible port scanner that queries Shodan's index instead of scanning directly — zero-touch enumeration when direct port scanning is against program rules.

```
$ smap -iL hosts.txt -oX shodan_ports.xml
```

aquatone

github.com/michenriksen/aquatone

PROBE

INSTALLED

Screenshots every live host for rapid visual triage of a large asset list — turns 500+ hosts into a browsable gallery to spot exposed admin panels and default install pages fast. Upstream has no go.mod and last committed in 2019, so a plain go install fails against a newer xurls API — built here by pinning mvdan.cc/xurls/v2@v2.0.0, the version its original Gopkg.toml specified, where the API still matches the code.

```
$ cat live.txt | aquatone -out aquatone_report/
```

eyewitness

github.com/RedSiege/EyeWitness

PROBE

INSTALLED

Alternative screenshot and reporting tool that also captures response headers/certs — backup for aquatone when headless Chrome has rendering issues on a target.

```
$ eyewitness --web -f live.txt -d eyewitness_out
```

katana

github.com/projectdiscovery/katana

CRAWL

INSTALLED

Modern web crawler with JS-rendering support — builds the full URL/form/parameter map for each live app, including JS-heavy SPAs, that every vuln-hunting phase tests against.

```
$ katana -u https://target.com -jc -d 3 -o katana_urls.txt
```

gau

github.com/lc/gau

CRAWL

INSTALLED

Pulls known URLs for a domain from the Wayback Machine, Common Crawl, AlienVault OTX, and URLScan — recovers historical and removed admin/debug endpoints active crawling won't find.

```
$ gau target.com | tee gau_urls.txt
```

waybackurls

github.com/tomnomnom/waybackurls

CRAWL

INSTALLED

Focused single-source Wayback Machine CDX puller — cheap, fast historical-URL sweep on a freshly-discovered target.

```
$ waybackurls target.com > wayback_urls.txt
```

waymore

github.com/xnl-h4ck3r/waymore

CRAWL

INSTALLED

Extended Wayback/CDX harvester that also retrieves archived response bodies — finds secrets or endpoints in pages that no longer resolve live.

```
$ waymore -i target.com -mode U -oU waymore_urls.txt
```

hakrawler

github.com/hakluke/hakrawler

CRAWL

INSTALLED

Fast, simple crawler for a quick pass on rate-limited targets, where katana's heavier JS rendering would trip WAF rate thresholds.

```
$ echo https://target.com | hakrawler -d 2
```

gospider

github.com/jaeles-project/gospider

CRAWL

INSTALLED

Crawler that also mines robots.txt/sitemap.xml and cross-domain links, filling gaps katana/hakrawler occasionally miss.

```
$ gospider -s https://target.com -o gospider_out -c 10 -d 2
```

cariddi

github.com/edoardottt/cariddi

CRAWL

INSTALLED

Crawler that flags secrets, endpoints, and juicy file extensions inline while crawling — a fast single-pass triage before dedicated secret scanners run.

```
$ echo https://target.com | cariddi -intensive -s
```

ffuf

github.com/ffuf/ffuf

FUZZ

INSTALLED

Fast, flexible HTTP fuzzer — the primary directory/file/vhost fuzzer used constantly to find hidden admin panels, backup files, and unlinked API routes.

```
$ ffuf -u https://target.com/FUZZ -w wordlist.txt -mc 200,301,403
```

feroxbuster

github.com/epi052/feroxbuster

FUZZ

INSTALLED

Rust-based recursive content-discovery fuzzer — used when automatic recursion into nested directories is preferred over ffuf's flat, single-level speed.

```
$ feroxbuster -u https://target.com -w wordlist.txt -x php,html
```

gobuster

github.com/OJ/gobuster

FUZZ

INSTALLED

Long-standing standard directory/DNS/vhost bruteforcer — fallback fuzzer and the go-to for DNS subdomain brute-force mode.

```
$ gobuster dir -u https://target.com -w wordlist.txt
```

arjun

github.com/s0md3v/Arjun

PARAM

INSTALLED

Discovers hidden/unlinked HTTP GET/POST parameters via wordlist probing. Hidden parameters are frequently the entire vulnerability — arjun surfaces the ones never referenced in the visible UI that later become IDOR, SSRF, or injection points.

```
$ arjun -u https://target.com/api/endpoint -oT params.txt
```

x8

github.com/Sh1Yo/x8

PARAM

INSTALLED

Rust alternative to Arjun for hidden-parameter discovery, used when Python performance is too slow against a large parameter wordlist or many target URLs.

```
$ x8 -u https://target.com/api -w params_wordlist.txt
```

gf

github.com/tomnomnom/gf

FILTER

INSTALLED

Pattern-matching grep wrapper with curated vulnerability-class patterns (XSS, SSRF, SQLi sinks) — filters huge crawled-URL lists down to the parameters that actually look injectable.

```
$ cat all_urls.txt | gf ssrf
```

qsreplace

github.com/tomnomnom/qsreplace

FILTER

INSTALLED

Replaces query-string values across a list of URLs with a payload placeholder — rewrites thousands of crawled URLs in one pass and pipes them straight into a scanner.

```
$ cat urls.txt | qsreplace 'FUZZ' | nuclei -t xss/
```

anew

github.com/tomnomnom/anew

FILTER

INSTALLED

Appends only new/unique lines to a file — the deduplication glue that keeps continuous recon monitoring additive instead of reprocessing the same hosts every run.

```
$ subfinder -d target.com -silent | anew subs_all.txt
```

dnsreaper

github.com/punk-security/dnsReaper

TAKEOVER

INSTALLED

Subdomain-takeover scanner supporting 50+ provider fingerprints. A dangling CNAME to a deprovisioned Heroku/S3/GitHub Pages resource is a full account or content takeover — one of the highest-signal, lowest-effort vulnerability classes.

```
$ dnsreaper scan --file subs.txt
```

subjack

github.com/haccer/subjack

TAKEOVER

INSTALLED

Fast Go-based takeover fingerprint scanner used as a lightweight fallback when dnsreaper's fingerprint database doesn't cover a specific provider.

```
$ subjack -w subs.txt -t 50 -o subjack_out.txt -ssl
```

graphw00f

github.com/dolevf/graphw00f

GRAPHQL

INSTALLED

Fingerprints which GraphQL engine (Apollo, Hasura, Graphene, etc.) a target endpoint runs — the first step of any GraphQL assessment, since the engine dictates which introspection tricks and known CVEs apply.

```
$ graphw00f -d -t https://target.com/graphql
```

clairvoyance

github.com/nikitastupin/clairvoyance

GRAPHQL

INSTALLED

Reconstructs a GraphQL schema via field-suggestion error messages even when introspection is disabled — defeats the single most common GraphQL hardening step.

```
$ clairvoyance -o schema.json https://target.com/graphql
```

PHASE 3 — VULNERABILITY ASSESSMENT (8 TOOLS)

Automated and semi-automated checks against the mapped surface for known CVEs, misconfigurations, weak authentication tokens, and WAF posture — triage before manual exploitation effort is spent.

nuclei

github.com/projectdiscovery/nuclei

SCAN

INSTALLED

Template-driven vulnerability scanner covering thousands of CVEs, misconfigs, and exposed panels. The core automated sweep run against every live host before manual testing begins.

```
$ httpx -l live.txt -silent | nuclei -t cves/ -severity high,critical -o nuclei_out.txt
```

semgrep

github.com/semgrep/semgrep

SAST

INSTALLED

Static analysis engine that flags insecure code patterns across languages via rulesets — used when source code is in scope (CI/CD assessments, disclosed repos) to catch injection sinks and hardcoded secrets at scale.

```
$ semgrep --config p/security-audit src/
```

log4j-scan

github.com/fullhunt/log4j-scan

CVE

INSTALLED

Focused active scanner for Log4Shell (CVE-2021-44228) and its bypass variants — legacy enterprise stacks still run vulnerable Log4j versions worth a dedicated targeted check.

```
$ python3 log4j-scan.py -u https://target.com --run-all-tests
```

graphql-cop

github.com/dolevf/graphql-cop

GRAPHQL

INSTALLED

Automated GraphQL security checklist covering the OWASP GraphQL testing guide (introspection, batching, field suggestion, CSRF) — ensures no standard check is skipped before manual work.

```
$ graphql-cop -t https://target.com/graphql
```

jwt_tool

github.com/ticarpi/jwt_tool

JWT

INSTALLED

Comprehensive JWT security tester — every JWT-authenticated target gets run through this for alg:none, RS256-to-HS256 key confusion, and weak-secret brute forcing.

```
$ python3 jwt_tool.py <token> -M at -pk public_key.pem
```

byp4xx

github.com/lobuhi/byp4xx

BYPASS

INSTALLED

Automated 403/401 bypass tester across header, path-encoding, and method tricks — turns manual bypass guesswork into one automated pass on any Forbidden endpoint.

```
$ byp4xx -u https://target.com/admin
```

whatwaf

github.com/Ekultek/WhatWaf

BYPASS

INSTALLED

Detects and fingerprints which WAF/CDN protects a target, so encoding and evasion payloads can be chosen for that specific vendor instead of guessed blind.

```
$ whatwaf -u https://target.com
```

unwaf

github.com/mmarting/unwaf

BYPASS

INSTALLED

Resolves the real origin IP behind a WAF/CDN using historical DNS and certificate data — once the vendor is known via whatwaf, this looks for a direct path that skips its filtering.

```
$ unwaf -d target.com
```

PHASE 4 — EXPLOITATION (11 TOOLS)

Turning a suspected weakness into a proven, reproducible impact — injection, XSS, file-upload RCE, credential attacks, and blind-vulnerability confirmation via out-of-band callbacks.

dalfox

github.com/hahwul/dalfox

XSS

INSTALLED

Fast XSS scanner with parameter mining, DOM analysis, and blind-XSS support — the primary automated tool for confirming reflected/DOM XSS with working payloads, callback included.

```
$ cat params.txt | dalfox pipe --blind https://your-server.oastify.com
```

xsstrike

github.com/s0md3v/XSSStrike

XSS

INSTALLED

Context-aware XSS detection/exploitation engine with WAF fingerprinting — used against heavily-filtered inputs where dalfox's payload set gets blocked outright.

```
$ python3 xsstrike.py -u 'https://target.com/search?q=test' --crawl
```

ghauri

github.com/r00th3x49/ghauri

SQLI

INSTALLED

Modern SQL injection tool built for speed on boolean/time-based blind SQLi — faster than sqlmap when request overhead makes exploitation impractically slow.

```
$ ghauri -u 'https://target.com/item?id=1' --dump
```

sqlmap

github.com/sqlmapproject/sqlmap

SQLI

INSTALLED

The standard automated SQL injection framework — detection, DBMS fingerprinting, data extraction, and often OS command execution via stacked queries, once SQLi is even suspected.

```
$ sqlmap -u 'https://target.com/item?id=1' --batch --dbs
```

fuXPloider

github.com/almandin/fuXPloider

UPLOAD

INSTALLED

Automated file-upload vulnerability scanner/exploiter — runs the full bypass matrix (double extensions, null bytes, content-type spoofing, polyglot files) to confirm code execution.

```
$ fuXPloider --url https://target.com/upload --not-regex "invalid"
```

hashcat

hashcat.net/hashcat

CRED

INSTALLED

GPU-accelerated password/hash cracking engine — cracks recovered hashes (dumped DB hashes, cracked JWT HMAC secrets) and mutates wordlists via its rule engine for spray attacks.

```
$ hashcat -m 0 -a 0 hashes.txt rockyou.txt -r best64.rule
```

cewler

github.com/roys/cewler

CRED

INSTALLED

Crawls a target site to build a custom wordlist from its own on-page vocabulary — employees reuse product names and internal project terms in passwords more often than you'd expect.

```
$ cewler https://target.com -o cewler_wordlist.txt
```

cupp

github.com/Mebus/cupp

CRED

INSTALLED

Interactive profiler that turns known facts about a specific employee (name, birthdate, pet, company) into a small high-probability password candidate list for a targeted spray.

```
$ cupp -i
```

trevorspray

github.com/blacklanternsecurity/TREVOrspray

CRED

INSTALLED

Password-spray orchestrator with OAuth/O365/Okta module support and jitter/delay controls — drives the actual credential spray against enterprise SSO endpoints with lockout-avoidance timing.

```
$ trevorspray -u users.txt -p passwords.txt --mod o365
```

kerbrute

github.com/ropnop/kerbrute

CRED

INSTALLED

Fast Kerberos pre-auth username enumeration and password spraying — on AD-integrated targets, enumerates valid domain usernames without generating failed-logon events an LDAP/SMB spray would trigger.

```
$ kerbrute passwordspray -d target.com userlist.txt Password123
```

interactsh-client

github.com/projectdiscovery/interactsh

OOB

INSTALLED

Self-hosted out-of-band interaction server client — confirms blind SSRF, blind XXE, and blind RCE by catching the DNS/HTTP callback the payload triggers, the only way to prove impact when the response gives no visible feedback.

```
$ interactsh-client -v # embed the generated domain in the payload
```

PHASE 5 — MOBILE RUNTIME & POST-EXPLOITATION (2 TOOLS)

Runtime instrumentation of mobile applications once static analysis is exhausted — defeating SSL pinning and driving live traffic capture for the same API-testing playbook used against the web target.

mobsf

github.com/MobSF/Mobile-Security-Framework-MobSF

MOBILE

INSTALLED

All-in-one mobile app security framework running automated static and dynamic analysis in one shot — a baseline report (permissions, manifest issues, hardcoded secrets, binary protections) before manual runtime testing begins.

```
$ mobsf # starts the local server on :8000; upload the APK/IPA via the web UI
```

objection

github.com/sensepost/objection

MOBILE

INSTALLED

Runtime mobile instrumentation toolkit built on Frida — patches a target APK to disable SSL certificate pinning at runtime, the prerequisite for proxying mobile traffic through Burp for the rest of the API-testing playbook.

```
$ objection patchapk -s target.apk
```